2.2 PPT

Tool Runtime 设计：让 Tool Call 进入受控执行

一、Runtime 是什么?

1.1 什么是 Tool Runtime，它有什么用?

Tool Runtime 是 Agent 应用中的工具执行层。它位于“模型返回 Tool Call”和“业务 handler 产生真实副作用”之间，负责把模型提出的候选动作，转换成一次可以校验、可以拒绝、可以执行、可以追踪的业务调用。

把它放回整条链路里：

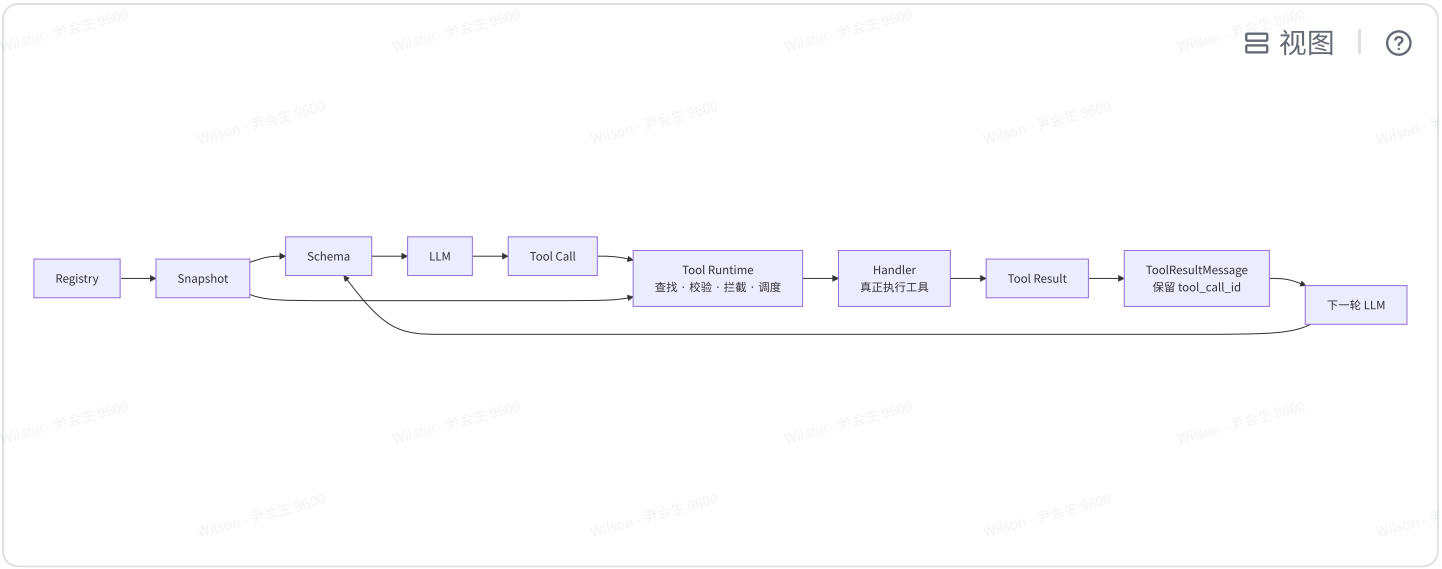
代码块

```
模型：我想调用 create_ticket，参数是……
          ↓
Tool Runtime：工具存在吗？参数合法吗？有权限吗？需要批准吗？依赖可用吗？
          ↓
handler：真正访问数据库、HTTP API 或文件系统
          ↓
Tool Runtime：校验结果、记录审计、保留 tool_call_id
          ↓
Agent Loop：把 ToolResultMessage 写回上下文，决定是否继续
```

核心概念：Runtime 不是模型，也不是业务 handler，更不是 Agent Loop 本身。

为什么必须有这一层?

1. 因为模型输出具有不确定性，而工具执行可能产生真实副作用。
2. Runtime 的价值，就是把这些问题放到业务副作用之前处理。



其中：

- `Registry`：系统有哪些工具。
- `Snapshot`：本轮实际开放哪些工具。
- `Schema`：模型能看到什么。
- `Runtime`：这次调用是否允许执行，以及如何执行。
- `ToolResultMessage`：把结果和原始 `tool_call_id` 重新关联，再交给下一轮 LLM。

> 学习重点不是背出某个框架的 Runtime 源码，而是能判断这条执行边界是否存在、是否放在正确的位置，以及它是否真的阻止了错误和越权。

1.2 本讲学习目标？

层级	验收方式
能解释	不看稿说清 Registry、Snapshot、Runtime 和 handler 分别解决什么问题
能补全	根据接口补出 Runtime 的查找、校验、拦截、执行和回写
能审查	找出 AI 代码中的越权、版本漂移、错误重试和 ID 丢失
能迁移	把工单示例迁移成数据库、HTTP 或文件工具，并重新设计权限、风险和失败语义

1.3 从 Function Calling 到 Tool Runtime？

上一节我们讲完了 Function Calling。模型读到工具的名称、说明和参数 Schema 以后，可以返回一条 Tool Call，告诉系统它想调用哪个工具，以及准备传入什么参数。

到这里会产生第二个问题：既然工具名和参数都有了，为什么不直接执行？

代码块

```
if call.name == "get_weather":
    result = await get_weather(**call.arguments)
elif call.name == "search_orders":
    result = await search_orders(**call.arguments)
elif call.name == "create_ticket":
    result = await create_ticket(**call.arguments)
```

生产环境中真正的问题：

call.name 是不是本轮开放的工具？

参数是否属于当前版本？

当前用户有没有权限？

下游服务不可用时怎么办？

工具执行超时以后能不能重试？

结果又怎样与原来的 Tool Call 对应？

… …

为此建立三个核心对象：

代码块

```
ToolDefinition：描述一个工具的协议、实现和治理规则
ToolRegistry：保存已注册工具，并为当前轮次生成工具快照
ToolRuntime：从快照中查找、校验、拦截、执行并包装结果
```

时间线：

代码块

```
工程师注册 ToolDefinition
          ↓
```

```
Harness 根据任务和策略发现 active tools，并冻结本轮快照
        ↓
从快照投影出模型可见的 Tool Schema
        ↓
模型返回 Tool Call
        ↓
Tool Runtime：prepare → execute → finalize
        ↓
ToolResultMessage 写回 Agent Loop，模型继续决策
```

先注册，再发现，最后执行。

二、先划清边界：Tool Schema、Registry 和 Runtime 分别解决什么问题？

为什么大家听 Tool、Function Calling、Registry、Runtime、Harness 时容易混乱？

因为这些词不在同一个层次，而且不同框架的命名也不完全一致。

下面这张表不用背，它是后文的索引。

名称	本节中的准确含义	容易混淆的地方
Function Calling / Tool Use	模型根据工具说明生成 Tool Call 的协议能力	它生成调用建议，不负责执行函数
Tool	提供给模型选择的一项受管理能力，通常由契约、治理信息和执行入口共同构成	这里的 Tool 不等于 Python 函数，也不等于一条 Tool Call
Tool Schema	真正发给模型的 name + description + parameters；不同 Provider 再适配成各自格式	它是 ToolDefinition 的安全投影，不包含 handler、权限和密钥
ToolDefinition	本课程在应用侧使用的完整工具定义	它包含模型协议、治理字段和 handler
AgentTool	pi 中对可执行工具接口的命名	是 pi 的类型名，不是行业统一标准
handler	查询数据库、调用 HTTP API 等具体业务函数	handler 只做业务动作，不等于完整 Tool
Tool Registry	保存已注册工具及版本的目录	管“有哪些”，不执行 Tool Call

工具注册	工程师把一份经过检查的 ToolDefinition 放入 Registry	注册成功只表示平台认识它，不表示本轮开放或已经执行
工具发现	Harness 根据任务、策略和运行模式筛选本轮候选工具	发生在模型请求之前，不等于模型选工具
active tools	工具发现产生的本轮开放集合	是发现结果，不是 Registry 的全部工具
ToolSnapshot	某一轮冻结后的工具定义视图	不是普通缓存，也不是数据库快照
Tool Runtime	Agent 应用中的工具执行层	不是 Python Runtime 或 Java Runtime
Harness	装配模型、工具、上下文、Hook 和本轮状态的外层运行框架	它组织运行，但不能绕过 Runtime 的检查
Agent Loop	请求模型、处理 Tool Call、写回结果并继续下一轮的控制循环	Loop 驱动流程，handler 不驱动流程
Tool Call	模型输出的工具名、参数和调用 ID	它是候选动作，不表示业务已经发生
ToolResult	handler 或 Runtime 内部产生的执行结果	还不一定能直接写回模型消息
ToolResultMessage	带调用关联信息、可以写回 Agent Loop 的结果消息	必须保留原来的 tool_call_id

从职责上再压缩一次：

代码块

```
Tool Schema / ToolDefinition：分别描述模型可见契约，以及完整契约与治理信息
Tool Registry：管理系统有哪些工具、有哪些版本
工具发现：决定本轮开放哪些工具
ToolSnapshot：冻结本轮模型协议与执行实现
Tool Runtime：决定这次能否执行、怎样执行和回写
handler：只完成具体业务动作
Harness / Agent Loop：装配本轮上下文并驱动多轮循环
```

还有谁这么干？ Pi 框架的源码也体现了这个边界。它的 AgentTool 在模型协议字段之外，保存真实执行入口和调度信息：

代码块

```
interface AgentTool<TParameters, TDetails> extends Tool<TParameters> {
```

```
  label: string;
  prepareArguments?: (args: unknown) => TParameters;
  execute(
    toolCallId: string,
    params: TParameters,
    signal?: AbortSignal,
    onUpdate?: (partialResult: AgentToolResult<TDetails>) => void,
  ): Promise<AgentToolResult<TDetails>>;
  executionMode?: "sequential" | "parallel";
}
```

这段接口要分成两部分看。

继承自 Tool 的名称、说明和参数 Schema 会投影给模型；

execute()、prepareArguments 和 executionMode 只在模型返回 Tool Call 以后由运行环境使用。

边界确定以后，我们按真实运行顺序讲三件事：

第一，怎样注册一份完整工具定义；

第二，怎样发现并冻结本轮工具；

第三，Runtime 怎样执行一条 Tool Call。

三、第一阶段——注册

3.1 为什么不能只注册一个 handler?

结论：handler 只回答“业务动作怎样完成”

如果 Registry 里只保存函数，代码可能是这样：

代码块

```
registry["create_ticket"] = create_ticket
```

更稳妥的做法，是注册一份不可变的 ToolDefinition。

代码块

```
@dataclass(frozen=True)
```

```
class ToolDefinition:
    # 第一组：调用契约
    name: str
    version: str
    description: str
    input_model: type[BaseModel]
    output_model: type[BaseModel]
    error_codes: tuple[str, ...]

    # 第二组：运行治理
    permission: str
    risk: Literal["low", "medium", "high"]
    timeout_s: float
    retry: RetryPolicy
    audit_fields: tuple[str, ...]
    dependencies: tuple[str, ...]
    execution_mode: Literal["sequential", "parallel"]

    # 第三组：真实实现
    handler: ToolHandler
```

这段代码而是建立一个检查框架。

第一组定义调用双方的契约；

第二组告诉 Runtime 怎样治理；

第三组才是真正访问数据库、HTTP API 或文件系统的业务函数。

比如，模型通常只需要名称、说明和输入参数：

代码块

```
def to_provider_tool(tool: ToolDefinition) -> dict[str, Any]:
    return {
        "type": "function",
        "function": {
            "name": tool.name,
            "description": tool.description,
            "parameters": tool.input_model.model_json_schema(),
        },
    }
```

3.2 一份完整 ToolDefinition 为什么要包含全部字段？

先统一一下名词：本节统一把真正发送给模型的 name + description + parameters 称为“模型可见 Tool Schema”。

输出、错误、权限、风险、超时、重试和审计属于运行侧 ToolDefinition，不应全部暴露给模型。

不同 Provider 的外层字段差异由适配层处理，不改变这条边界。

3.3 Tool Registry 注册时要做哪些检查？

工程角度：**注册阶段负责尽早发现定义冲突，不负责决定本轮是否开放，也不负责执行工具。**

代码块

```
class ToolRegistry:
    def __init__(self) -> None:
        self._tools: dict[tuple[str, str], ToolDefinition] = {}

    def register(self, tool: ToolDefinition) -> None:
        key = (tool.name, tool.version)
        if key in self._tools:
            raise ValueError(f"duplicate tool: {tool.name}@{tool.version}")
        if tool.timeout_s <= 0 or tool.retry.max_attempts < 1:
            raise ValueError("timeout and retry policy must be positive")
        self._tools[key] = tool

    def get(self, name: str, version: str) -> ToolDefinition:
        try:
            return self._tools[(name, version)]
        except KeyError as exc:
            raise KeyError(f"tool not registered: {name}@{version}") from exc
```

生产环境在这里检查名称格式、Schema 是否可生成、错误码是否重复、超时是否为正数，以及依赖声明是否完整。

注册成功只代表平台认识这个工具，是否开放给当前任务，是下一阶段“工具发现”要解决的问题。

3.4 工具元数据和分类应该怎样设计？

这一小节只保留一个判断标准：每个元数据字段都要有明确消费者。

代码块

```python
@dataclass(frozen=True)
class BusinessMetadata:
    category: Literal["database", "http", "file", "external"]
    access: Literal["read", "write"]
    owner: str
    data_classification: Literal["public", "internal", "sensitive"]
```

这段代码的作用，是让策略可以读取元数据。

注册阶段到这里结束。我们已经有了“平台认识的全部工具”，但还没有把它们全部发给模型。

下一步要做的是从全量工具中选出本轮真正需要的集合。

四、第二阶段——发现：本轮模型应该看到哪些工具？

4.1 怎么定义注册、发现和执行？

代码块

```
注册：平台是否认识这项工具
发现：当前 Agent、任务和模式是否应该把它交给模型
执行：当前用户的这一次 Tool Call 是否真的允许发生
```

例如，平台已经注册 search_orders 和 create_ticket。客服只读模式只发现前者，工单处理模式才发现两者。

工具发现发生在模型请求之前，由 Harness 决定候选集合；模型选工具发生在请求之后，由 Function Calling 在候选集合中生成某个 Tool Call。前者决定模型**可以选什么**，后者决定模型**这一次想选什么**。

代码块

```python
TOOL_PROFILES = {
    "customer_readonly": {
        "search_orders": "1.0.0",
    },
    "ticket_operation": {
        "search_orders": "1.0.0",
        "create_ticket": "1.0.0",
    },
}
```

```
10
11  routes = TOOL_PROFILES[run_mode]
12  snapshot = registry.snapshot(routes, enabled_state)
```

这段代码里，routes 同时明确了工具名和版本。

4.2 为什么模型和 Runtime 必须使用同一份快照?

为了保证模型看到的协议与 Runtime 最终执行的实现是同一份定义。采用下面的定义方法

代码块

```
1   snapshot = registry.snapshot(routes, enabled_state)
2   provider_tools = snapshot.provider_tools()
3
4   assistant_message = await provider.complete(
5       messages=messages,
6       tools=provider_tools,
7   )
8
9   for raw_call in parse_tool_calls(assistant_message):
10      result_message = await runtime.invoke(
11          snapshot=snapshot,
12          call=ToolCall.model_validate(raw_call),
13          context=context,
14      )
15      messages.append(result_message)
```

这段代码有一个刻意的设计：模型请求和 runtime.invoke() 使用同一个 snapshot。

Snapshot ≠ 缓存！ 本轮 Snapshot 的职责是“**不更新!**”，具体逻辑是：

代码块

```
1   T1: Harness 从 Registry 解析出 v1，并把 v1 Schema 发给模型
2   T2: 管理员把默认版本切换到 v2
3   T3: 模型仍按刚才看到的 v1 Schema 返回 status="pending"
4   T4: 如果 Runtime 重新读取 Registry，就会拿 v2 handler 执行 v1 参数
```

4.3 工具版本和灰度切换应该放在哪里？

一个生产问题：**工具已经在运行，怎么安全地升级？**

假设我们有一个 `search_orders` 工具。v1 查询速度较慢，准备上线 v2，v2 增加了订单状态过滤，但内部 SQL 和返回字段也发生了变化。

所以，Runtime 需要考虑版本路由和灰度切换。

代码块

```
def choose_version(
    tool_name: str,
    tenant_id: str,
    stable: dict[str, str],
    canary: dict[str, tuple[str, set[str]]],
) -> str:
    candidate = canary.get(tool_name)

    if candidate and tenant_id in candidate[1]:
        return candidate[0]

    return stable[tool_name]


routes = {
    name: choose_version(
        tool_name=name,
        tenant_id=context.tenant_id,
        stable=stable_versions,
        canary=canary_versions,
    )
    for name in requested_tools
}

snapshot = registry.snapshot(routes)
```

一个完整的升级过程通常是：

1. 注册 v2，但不立即替换 v1；
2. 只让测试租户使用 v2；

3. 对比成功率、延迟、错误类型和结果质量；
4. 逐步扩大灰度范围；
5. 将 v2 切换为 stable 版本；
6. 等待旧会话和旧工作流完成后，再下线 v1。

版本切换应该发生在创建快照之前；快照一旦创建，本轮执行就必须使用同一版本。

4.4 工具启停和依赖检查为什么要分两次做？

发现阶段决定工具能不能被模型看到，执行阶段确认工具现在还能不能真正运行。

这两个检查必须分开，因为它们发生在不同时间点。

还是以 `search_orders` 为例。创建快照时，这个工具是正常的，所以它被交给模型。模型思考几秒后生成了 Tool Call，但就在这几秒内，订单服务出现故障，管理员把工具关闭。

启停状态应该独立于工具定义保存：

代码块

```
enabled_state = {
    "search_orders": True,
    "create_ticket": False,
}
```

创建快照时，先过滤掉已停用工具：

代码块

```
snapshot = registry.snapshot(
    routes=routes,
    enabled_state=enabled_state,
)
```

但 Runtime 执行 Tool Call 时，还要再次读取最新状态：

代码块

```
def execute_tool(call, snapshot, enabled_state):
    tool = snapshot.get(call.name)

    if tool is None:
        return error_message(
            call,
            "TOOL_NOT_FOUND",
            "tool is not available in this snapshot",
        )

    if not enabled_state.get(call.name, True):
        return error_message(
            call,
            "TOOL_DISABLED",
            "tool is temporarily disabled",
        )

    return tool.handler(**call.arguments)
```

这里的两次检查分别解决两个问题：

- 快照检查：减少模型提出无效工具调用；
- 执行前检查：保证管理员可以在运行中立即阻止危险或异常调用。

依赖检查也可以按这个思路分层。

在工具注册或服务启动时，检查静态依赖，例如 API 密钥、数据库客户端和本地目录是否存在：

代码块

```
def validate_registration(config):
    if not config.api_key:
        raise ValueError("missing api key")

    if not config.data_dir.exists():
        raise ValueError("data directory does not exist")
```

执行前再检查动态依赖，例如数据库是否健康、内部 HTTP 服务是否可访问：

```
def check_runtime_dependency(tool, call):
    if tool.requires_db and not db.is_healthy():
        return error_message(
            call,
            "DEPENDENCY_UNAVAILABLE",
            "required dependency is temporarily unavailable",
        )

    return None
```

通用做法：

- 注册阶段检查基础配置；
- 创建快照时过滤停用工具；
- 执行阶段只检查当前工具需要的依赖；
- 异常时返回稳定错误码，不泄露内部地址、密钥名称和堆栈信息。

到这里，我们就能看到 Registry、Snapshot 和 Runtime 的分工：

- Registry 保存工具定义和版本；
- Snapshot 冻结本轮可见工具及其版本；
- Runtime 在执行前重新确认状态、权限和依赖。

五、第三阶段——执行：一条 Tool Call 在 Runtime 中怎样流动？

5.1 先分清参与者

参与者	负责什么	不负责什么
模型	从开放工具中提出名称和候选参数	不决定权限、风险、版本和批准状态
Agent Loop	请求模型、保存消息、把 Tool Call 交给 Runtime、决定是否继续	不直接实现业务动作
Tool Runtime	查找、校验、拦截、调度、归一化错误和包装结果	不替 handler 实现业务语义
handler	查询订单、调用内部 API、处理文件等具体动作	不自行决定模型可见范围

工程师	设计以上边界和策略	不把治理决策交给模型猜

分层的直接收益是，每个失败都能停在正确位置。工具名不在快照里，handler 不会被找到；参数非法，handler 不会启动；权限不足，业务系统不会收到请求；handler 抛异常，Agent Loop 也不会被 Python 堆栈打断。

5.2 Runtime 为什么要分成 prepare、execute 和 finalize?

总结构：

代码块

```
prepare
  查找工具 → 参数兼容 → Schema 校验 → 启停/权限/风险/依赖检查
        ↓
execute
  调用 handler → 超时 → 取消 → 按策略重试 → 过程更新
        ↓
finalize
  输出校验 → 后置脱敏 → 审计 → ToolResultMessage → 写回 Agent Loop
```

prepare 保证“不该执行的调用绝不进入业务系统”；

execute 控制“业务动作怎样运行”；

finalize 保证“无论成功失败，都产生可关联、可消费的结果”。

5.3 Runtime 到底做了什么?

假设模型返回：

代码块

```
{
  "id": "call-42",
  "name": "create_ticket",
  "arguments": {
    "title": "退款失败",
    "description": "用户已等待两天",
    "priority": "high"
  }
}
```

Runtime 收到以后，依次处理：

第一步，从本轮 snapshot 查找 create_ticket。

第二步，用 CreateTicketInput 校验 arguments。

第三步，从认证后的 ExecutionContext 读取 user_id、tenant_id 和权限，而不是相信模型参数。

第四步，检查 ticket-service 和凭据是否可用。

第五步，调用 handler，并应用超时与重试规则。

第六步，校验 handler 输出，执行脱敏和审计，最后创建带有 tool_call_id="call-xxx" 的 ToolResultMessage。Agent Loop 依据这个 ID，把结果与原 Tool Call 对应起来，再交给模型继续回答。

5.4 核心的 invoke() 应该怎样组织?

Runtime 的骨架：

代码块

```
async def invoke(
    self,
    snapshot: ToolSnapshot,
    call: ToolCall,
    context: ExecutionContext,
) -> ToolResultMessage:
    tool = snapshot.tools.get(call.name)
    if tool is None:
        return self.error(call, "TOOL_NOT_FOUND")

    params = tool.input_model.model_validate(call.arguments)
    await self.policy.check(tool, call, params, context)

    raw_output = await asyncio.wait_for(
        tool.handler(params, context),
        timeout=tool.timeout_s,
    )
    output = tool.output_model.model_validate(raw_output)

    return self.finalize(tool, call, output, context)
```

这段代码位于主链的 prepare → execute → finalize 阶段，解决的是把不可信 Tool Call 变成受控业务执行。

5.5 如果 AI 生成了一段能运行的 Runtime，应该怎样审查?

如下代码有问题吗？

代码块

```
async def invoke(registry, call, context):
    tool = registry.get_latest(call.name)
    result = await tool.handler(**call.arguments)

    if tool.permission not in context.permissions:
        return {"error": "PERMISSION_DENIED"}
    if call.arguments.get("approved") is not True:
        return {"error": "APPROVAL_REQUIRED"}

    return {"result": result}
```

这段代码语法没有问题，正常参数下甚至会返回成功，但它至少有五个工程错误。

第一，它在执行时读取 Registry 的最新版本，没有使用本轮 Snapshot。模型看到 v1 Schema、Runtime 却执行 v2 handler 时，就会发生协议漂移。

第二，它把模型生成的原始参数直接传给 handler，没有经过 Schema 校验。多余字段、错误类型和越界值都会进入业务系统。

第三，它先执行 handler，再检查权限。即使最后返回 PERMISSION_DENIED，工单可能已经创建。错误响应不能撤销已经发生的副作用。

第四，它相信 arguments.approved。这相当于允许模型自己填写批准结果。批准记录必须来自可信 ExecutionContext，并与当前 tool_call_id 和参数绑定。

第五，它没有超时、稳定错误结构、输出校验、审计和 tool_call_id。并发调用以后，Agent Loop 无法可靠地把结果对应回原请求。

5.6 多个 Tool Call 什么时候并行，什么时候顺序执行？

一个效率问题，能不能并行？**并行不是模型决定的，而是工具定义和 Runtime 策略共同决定的。**

代码块

```
async def invoke_many(
    runtime: ToolRuntime,
    snapshot: ToolSnapshot,
    calls: list[ToolCall],
    context: ExecutionContext,
) -> list[ToolResultMessage]:
    must_run_in_order = any(
```

```
        snapshot.tools.get(call.name)
        and snapshot.tools[call.name].execution_mode == "sequential"
        for call in calls
    )

    if must_run_in_order:
        return [
            await runtime.invoke(snapshot, call, context)
            for call in calls
        ]

    return await asyncio.gather(
        *(runtime.invoke(snapshot, call, context) for call in calls)
    )
```

到这里，执行主线已经完整了。下一部分把注册、发现、快照和 Runtime 放进同一个 Python 文件，验证成功、参数错误、权限不足、未批准、内部异常归一化和临时下线六条路径。

六、完整 Python 演示：怎样从零搭出最小 Tool Runtime?

6.1 准备环境?

下面的代码使用 Python 3.11+ 和 Pydantic 2.x，不需要模型 API，也不访问真实网络。

Agent 的工具可以参考瑞幸咖啡或者麦当劳的接口

瑞幸咖啡：https://open.lkcoffee.com/docs

麦当劳：https://open.mcd.cn/mcp/doc

我们用一个假的内部 HTTP handler 模拟 create_ticket，目的是先把 Runtime 骨架跑通。

代码块

```
提示词：

基于 DeepSeek API 的可运行 Python 代码，实现一个最小但完整的 Agent Tool Runtime：
- 包含 ToolDefinition、ToolRegistry、ToolSnapshot、ToolSchema、ToolRuntime、Handler 和 ToolResultMessage；
- 支持注册天气查询、数据库查询两个工具，
- 完成“LLM → Tool Call → Tool Runtime → Handler → Tool Result → 下一轮 LLM”的闭环，
- 包含参数校验、权限检查、风险等级、超时、错误结构化、tool_call_id 保留、工具停用和基础日志；
- 代码应可直接运行，使用环境变量读取 DEEPSEEK_API_KEY，
```

```
- 提供完整依赖、示例工具、主程序和最小测试。

禁止：
- 不要把工具实现直接暴露给模型，
- 不要跳过 Registry、Snapshot、Schema 和 Runtime 直接调用 Handler；
- 不要省略参数校验、权限检查、结构化错误和 tool_call_id。
```

把代码保存为 tool_runtime_demo.py，然后执行：

代码块

```
python -m pip install "pydantic>=2,<3"
python tool_runtime_demo.py
```

运行结果应为：

代码块

```
模型可见工具: create_ticket
call-1 -> INVALID_ARGUMENT
call-2 -> PERMISSION_DENIED
call-3 -> APPROVAL_REQUIRED
call-4 -> OK {"status": "created", "ticket_id": "T-1001"}
call-5 -> INTERNAL_ERROR
call-6 -> TOOL_DISABLED
验收通过: 4 条拒绝路径未进入 handler，内部异常也没有泄漏 Python 堆栈
验收通过: 成功结果保留 tool_call_id，旧快照中的停用工具仍被拒绝
审计记录数: 6
```

6.2 这份代码应该怎样读？

检查点	解决什么	重点对象
ToolDefinition	把契约、治理和实现组成受管理工具	input_model、策略字段、handler、安全投影
Registry	管理平台有哪些 name@version	register()、组合键、重复检查
Snapshot	冻结本轮 Schema 与执行实现	routes、MappingProxyType、provider_tools()

Runtime.invoke	完成 prepare、execute、finalize	查找、校验、策略、handler、结果包装
测试与结果	用副作用探针证明边界有效	handler_calls、稳定错误、tool_call_id

6.3 接入真实 Provider 时，哪一段需要替换？

代码块

```
snapshot = registry.snapshot(routes, enabled)
assistant_message = await provider.complete(
    messages=messages,
    tools=snapshot.provider_tools(),
)

for raw_call in parse_tool_calls(assistant_message):
    call = ToolCall.model_validate(raw_call)
    result_message = await runtime.invoke(snapshot, call, context)
    messages.append(to_provider_result(result_message))
```

6.4 学习时应该先自己写，还是先让 AI 写？

在凑齐了第一个 Agent 必要的要素 （Agent = LLM + prompt + tool）之后，我们要考虑 AI 写代码的重要问题。

我的结论是：**先自己定义骨架和验收条件，再让 AI 补实现，最后自己做故障验证。**

比如：**写一个支持创建订单、搜索订单两个工具的 Agent 。**

第一遍练习时，先关掉演示代码，只写二十行左右的伪代码：注册 ToolDefinition、创建 Snapshot、从 Snapshot 查找工具、校验参数、检查权限、执行 handler、包装带 tool_call_id 的结果。这里不要求语法完整，但对象和顺序不能错。

第二步，再把接口、伪代码和约束交给 AI，让它补数据类、异常分支和测试样板。此时你给 AI 的不是一句“帮我写 Tool Runtime”，而是一份明确规格：哪些字段来自模型、哪些来自可信上下文、写操作不自动重试、所有拒绝路径不得进入 handler。

第三步，不看 AI 的解释，只看测试证据。故意传错参数、删除权限、伪造 approved=true、切换工具版本、执行前停用工具，确认 Runtime 在正确阶段停止。然后再把 create_ticket 换成 search_orders，重新决定权限、风险和重试策略。

如果你只能复制成功路径，还没有掌握；如果你能写出边界和测试，再让 AI 完成重复代码，这正是现代工程师应该具备的协作能力。

七、走向生产环境：哪些策略必须接回主链？

最后这部分不扩展新的架构，只回答四个生产问题：

1. 依赖坏了怎样提前失败，
2. 超时以后怎样决定是否重试，
3. 审计究竟记录什么，
4. Hook 应该放在哪个位置。

7.1 工具依赖怎样声明，检查结果又怎样影响执行？

依赖声明跟随 ToolDefinition，依赖检查分为加载检查和调用检查，失败必须发生在 handler 之前。

检查失败时，对模型返回稳定的 DEPENDENCY_UNAVAILABLE；依赖名称、内网地址和原始堆栈留在受控日志中。这样模型知道“当前工具暂不可用”，运维又能在日志里看到真实原因。

7.2 超时和重试为什么不能只写两个数字？

超时决定等待边界，重试决定是否再次产生副作用；两者必须结合工具语义设计。

查询天气和查询订单是读操作，遇到一次连接重置，通常可以退避后重试一到两次。

创建工单、支付和删除文件是写操作，超时只表示调用方没有及时收到结果，不代表下游没有成功。此时原样重试，可能重复创建、重复扣款或重复删除。

7.3 审计字段的作用是什么，哪些内容不能记录？

审计用于重建一次调用的决策和结果，不是把所有参数和返回值原样复制到日志。

一条可用审计记录应该能回答“谁在什么上下文中调用了哪个版本，Runtime 为什么放行或拒绝，最终发生了什么”。

代码块

```
audit_record = {
    "tool_call_id": call.id,
    "tool": tool.name,
    "version": tool.version,
    "tenant_id": context.tenant_id,
    "user_id": context.user_id,
    "outcome": result.error["code"] if result.error else "OK",
    "argument_fields": sorted(call.arguments.keys()),
```

```
}
```

7.4 before Hook 和 after Hook 应该各自放什么？

before Hook 适合权限、风险、人工批准、依赖和最新启停检查。

after Hook 适合输出校验后的脱敏、结果改写、审计补充和终止提示，但不能假装撤销已经发生的写操作。

到这里，生产策略已经全部回到主链：

1. 元数据和版本在注册阶段定义；
2. 分类、灰度和启停在发现阶段筛选；
3. 权限、风险和依赖在 prepare 阶段检查；
4. 超时和重试在 execute 阶段控制；
5. 输出校验、脱敏和审计在 finalize 阶段完成。

八、本节总结：学完 Tool Runtime，应该真正会什么？

你需要掌握三层、一条主链和一套验证方法。

第一层是 ToolDefinition。它描述调用契约、治理规则和真实 handler。

第二层是 Tool Registry 与 ToolSnapshot。

第三层是 ToolRuntime。它把不可信的 Tool Call 分成 prepare、execute 和 finalize：先查找与校验，再受控执行，最后统一包装 ToolResultMessage，并始终保留 tool_call_id。

在这条链路里，人和 AI 的分工也应该清楚：

工程师必须负责	AI 可以辅助
判断业务动作是否应该做成 Tool，工具粒度多大	生成数据类、序列化和样板代码
划分模型参数与可信上下文	根据明确 Schema 补实现
设计 Registry、Snapshot、Runtime 的职责	完成重复分支和普通单元测试
决定权限、风险、超时、重试和审计语义	根据既定规则重构代码

设计故障测试并审查结果	根据测试失败继续修改
对生产行为和副作用负责	提高实现效率

九、课堂练习

9.1 口头验收：能不能脱离代码讲清边界？

给自己两分钟，在白纸上画出“注册 → 发现 → 快照 → Tool Call → prepare → execute → finalize → Tool Result”的链路，然后解释下面四组区别：

- Tool Schema 与 ToolDefinition；
- Registry 与 Runtime；
- 工具发现与模型选工具；
- handler、Harness 与 Agent Loop。

9.2 代码练习：为代码增加异常处理？

不看完整代码，只根据接口补出 invoke() 的五个检查点：从 Snapshot 查找、参数校验、策略拦截、受控执行、结果包装。实现：

1. 非法参数不会进入 handler；
2. 权限不足不会产生业务副作用；
3. 模型伪造 approved=true 不能绕过批准；
4. 执行阶段不能读取 Registry 的最新版本；
5. 所有成功和失败结果都保留原 tool_call_id。
6. 写操作超时且没有幂等保证时不会被自动重试；
7. 未捕获的内部异常被归一化为稳定错误，返回中不包含 Python 堆栈。

建议使用 AI 补测试样板，但失败条件和断言必须由你先写清。